

Clase 3: Modules, Instanciación y Síntesis

Diseño e Implementación de Circuitos Digitales en VLSI

Agustín Pesce - José Torres - Santiago Garaventa — Becas de Grado 2026

Mayo 2026

Contents

1	Modules: la unidad de diseño	2
1.1	Anatomía de un module	2
1.2	Ports: la interfaz del module	2
1.3	Convenciones de nombres	2
1.4	Parámetros	3
2	Instanciación: conectando modules	3
2.1	Sintaxis básica	3
2.2	Ejemplo: Full Adder con Half Adders	4
2.3	Ejemplo: Ripple Carry Adder	5
2.4	Jerarquía de diseño	5
3	Generate: crear hardware con loops	5
3.1	El problema	5
3.2	generate for: la solución	6
3.3	Ejemplo: Barrel Shifter con generate	6
3.4	Ejemplo: Cadena de registros (shift register)	7
4	Diseño registrado: el sumador del laboratorio	8
4.1	¿Por qué registrar?	8
4.2	El sumador registrado	8
5	Primer contacto con síntesis	9
5.1	¿Qué es síntesis?	9
5.2	SDC: las constraints	9
5.3	Input Delay y Output Delay	10
5.3.1	Input Delay	10
5.3.2	Output Delay	10
5.3.3	En el SDC	10
6	Resumen	11

1 Modules: la unidad de diseño

En la clase anterior vimos `assign` y `always_ff`. Ahora vamos a ver cómo se organizan en **modules** y cómo se conectan entre sí para construir diseños complejos.

1.1 Anatomía de un module

Un module tiene cuatro partes: nombre, parámetros (opcionales), ports (entradas y salidas), y cuerpo (la lógica).

```
module contador #(
    parameter int WIDTH = 8          // 1. Parametros
) (
    input  logic          i_clk,      // 2. Ports
    input  logic          i_rst,
    input  logic          i_enable,
    output logic [WIDTH-1:0] o_count
);
    // 3. Senales internas
    logic [WIDTH-1:0] count_reg;

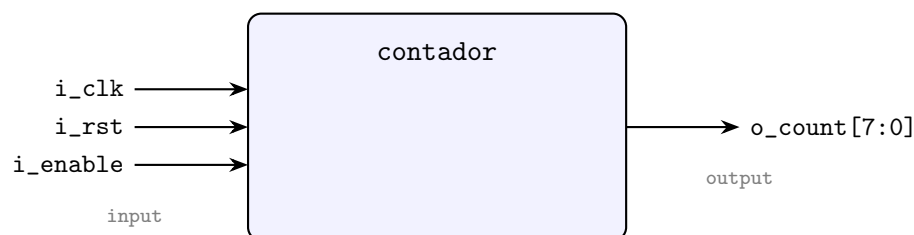
    // 4. Logica
    always_ff @(posedge i_clk)
        if (i_rst)          count_reg <= '0;
        else if (i_enable) count_reg <= count_reg + 1'b1;

    assign o_count = count_reg;
endmodule
```

1.2 Ports: la interfaz del module

Los ports definen cómo se comunica el module con el mundo exterior:

- **input**: señal que entra al module. El module la lee pero no la modifica.
- **output**: señal que sale del module. El module la genera.
- **inout**: bidireccional (raro, se usa casi exclusivamente para pads de I/O).



1.3 Convenciones de nombres

En la industria se usan convenciones para que el código sea legible:

Prefijo/Sufijo	Significado
i_	Input port
o_	Output port
_reg	Señal que es salida de un flip-flop
_next	Señal combinacional que va a la entrada D de un flip-flop
_n	Señal activa en bajo (negada)
WIDTH, DEPTH	Parámetros en MAYÚSCULAS

Estas convenciones no son obligatorias para el compilador, pero hacen que el código sea mucho más fácil de leer y mantener. Cuando ves `count_reg` sabés que es un flip-flop; cuando ves `count_next` sabés que es la lógica combinacional que alimenta al flip-flop.

Ejemplo del patrón `_reg` / `_next`:

```
logic [7:0] count_reg;    // flip-flop (estado actual)
logic [7:0] count_next;  // logica combinacional (proximo estado)

always_comb
    count_next = count_reg + 1'b1; // calcula el proximo valor

always_ff @(posedge i_clk)
    if (i_rst) count_reg <= '0;
    else      count_reg <= count_next; // captura el proximo valor
```

1.4 Parámetros

Los parámetros hacen el module **reutilizable**. En vez de hardcodear el ancho a 8 bits, lo hacés configurable:

```
module contador #(
    parameter int WIDTH = 8 // valor default
) (
    input  logic [WIDTH-1:0] i_data,
    output logic [WIDTH-1:0] o_data
);
```

El valor de `WIDTH` se define cuando instanciás el module:

```
contador #(.WIDTH(16)) cnt_16bit (...); // este tiene 16 bits
contador #(.WIDTH(32)) cnt_32bit (...); // este tiene 32 bits
contador                cnt_default (...); // este usa el default (8
    bits)
```

Cada instancia genera hardware **independiente** con el ancho especificado. Es como un template en C++ — el mismo código genera hardware diferente según el parámetro.

2 Instanciación: conectando modules

2.1 Sintaxis básica

Para usar un module dentro de otro, lo **instanciás**. La sintaxis es:

```
nombre_del_module #(.PARAM1(valor1)) nombre_instancia (
    .port1(signal1),
    .port2(signal2),
    ...
```

```
);
```

Cada `.port(signal)` conecta un port del module instanciado a una señal del module padre. El punto es obligatorio — indica que estás conectando por nombre.

2.2 Ejemplo: Full Adder con Half Adders

En la clase anterior vimos el half adder y el full adder. Ahora veamos cómo construir el full adder **instanciando** half adders:

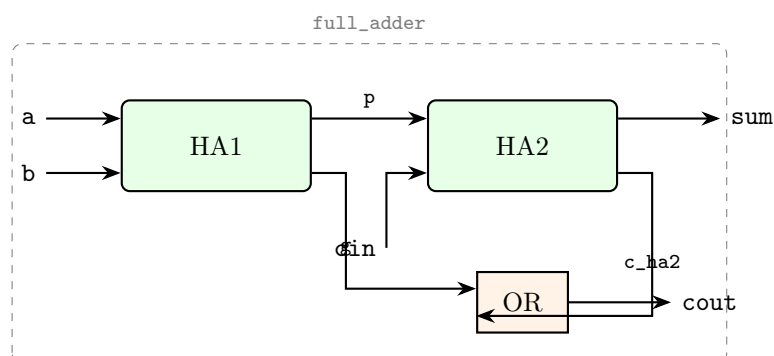
```
module half_adder (
    input  logic a, b,
    output logic sum, carry
);
    assign sum    = a ^ b;
    assign carry  = a & b;
endmodule
```

```
module full_adder (
    input  logic a, b, cin,
    output logic sum, cout
);
    logic p, g, c_ha2;    // senales internas (cables entre los HAs)

    // Instanciar primer half adder
    half_adder ha1 (
        .a(a),           // port a del HA -> signal a del FA
        .b(b),           // port b del HA -> signal b del FA
        .sum(p),          // port sum del HA -> cable interno p
        .carry(g)         // port carry del HA -> cable interno g
    );

    // Instanciar segundo half adder
    half_adder ha2 (
        .a(p),            // recibe la partial sum del primer HA
        .b(cin),          // recibe el carry de entrada
        .sum(sum),        // esta ES la sum final
        .carry(c_ha2)     // carry del segundo HA
    );

    // El carry out es el OR de los dos carries
    assign cout = g | c_ha2;
endmodule
```



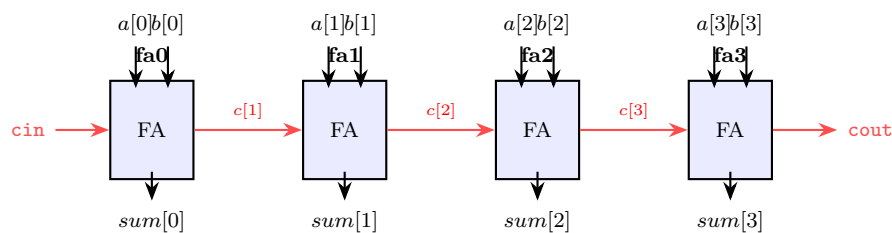
Notar que **p**, **g** y **c_ha2** son señales **internas** del full adder — cables que conectan los half adders entre sí. No son ports, no se ven desde afuera.

2.3 Ejemplo: Ripple Carry Adder

Ahora construimos un sumador de 4 bits instanciando 4 full adders:

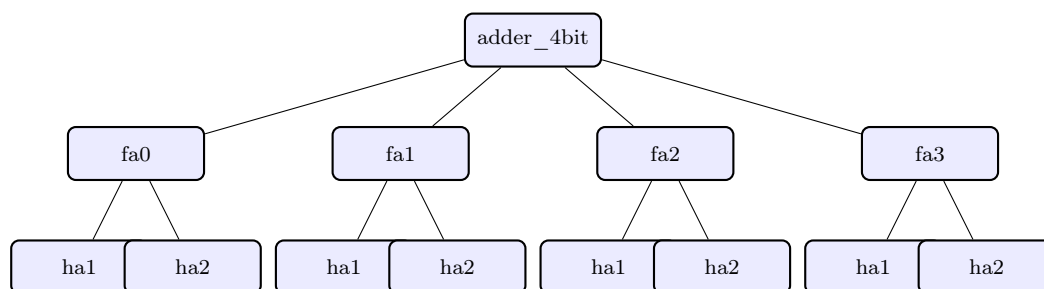
```
module adder_4bit (
    input  logic [3:0] a, b,
    input  logic      cin,
    output logic [3:0] sum,
    output logic      cout
);
    logic [3:1] c; // carries intermedios (c[1], c[2], c[3])

    full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout
        (c[1]));
    full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout
        (c[2]));
    full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout
        (c[3]));
    full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout
        (cout));
endmodule
```



Tenemos tres niveles de jerarquía: **adder_4bit** contiene 4 **full_adder**, cada uno contiene 2 **half_adder**. Eso es la jerarquía de diseño.

2.4 Jerarquía de diseño



Cada instancia es hardware **independiente**. **fa0.ha1** y **fa1.ha1** son dos half adders físicamente separados, con sus propios cables y compuertas. Compartir el nombre no comparte hardware.

3 Generate: crear hardware con loops

3.1 El problema

El ripple carry adder de 4 bits tiene 4 líneas casi idénticas:

```
full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .sum(sum[0]), .cout(c
    [1]));
full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c[1]), .sum(sum[1]), .cout(c
    [2]));
full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c[2]), .sum(sum[2]), .cout(c
    [3]));
full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c[3]), .sum(sum[3]), .cout(c
    cout));
```

Para 128 bits habría que escribir 128 líneas. Eso no escala.

3.2 generate for: la solución

`generate` permite crear hardware usando un loop que se resuelve en **tiempo de compilación** (no en runtime):

```
module adder_Nbit #(parameter int N = 8) (
    input  logic [N-1:0] a, b,
    input  logic         cin,
    output logic [N-1:0] sum,
    output logic         cout
);
    logic [N:0] c;          // N+1 carries
    assign c[0] = cin;      // carry inicial

    genvar i;
    generate
        for (i = 0; i < N; i++) begin : gen_fa
            full_adder fa (
                .a(a[i]), .b(b[i]), .cin(c[i]),
                .sum(sum[i]), .cout(c[i+1])
            );
        end
    endgenerate

    assign cout = c[N];     // carry final
endmodule
```

Importante

`generate for` no es un loop de software. No se ejecuta ciclo por ciclo. El compilador lo “desenrolla” y genera N instancias de hardware. Si N=128, genera 128 full adders con todas sus conexiones. El resultado es exactamente el mismo que escribir 128 líneas a mano.

Notar:

- `genvar i`: variable especial que solo existe en tiempo de compilación.
- `begin : gen_fa`: el label es obligatorio — cada iteración genera un bloque con nombre (`gen_fa[0]`, `gen_fa[1]`, ...).
- El `c[N:0]` tiene N+1 elementos para almacenar todos los carries (incluyendo `cin` y `cout`).

3.3 Ejemplo: Barrel Shifter con generate

Un barrel shifter desplaza un dato N posiciones a la izquierda. Cada etapa decide si desplazar 2^k posiciones o no, según el bit correspondiente de `shift_amount`:

```

module barrel_shifter #(parameter int WIDTH = 8) (
    input  logic [WIDTH-1:0]      i_data,
    input  logic [$clog2(WIDTH)-1:0] i_shift, // bits necesarios
    output logic [WIDTH-1:0]      o_data
);
    localparam int STAGES = $clog2(WIDTH);
    logic [WIDTH-1:0] stage [0:STAGES]; // unpacked array de etapas

    assign stage[0] = i_data; // la primer etapa es la entrada

    genvar k;
    generate
        for (k = 0; k < STAGES; k++) begin : gen_stage
            assign stage[k+1] = i_shift[k] ?
                (stage[k] << (1 << k)) : // shift 2^k posiciones
                stage[k];                // no shift
        end
    endgenerate

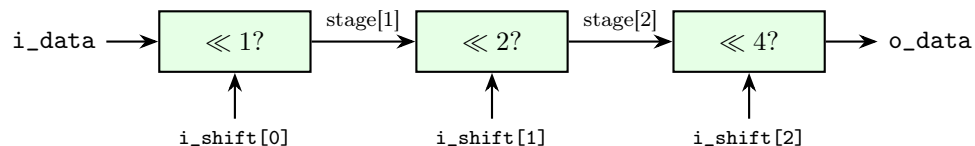
    assign o_data = stage[STAGES]; // la ultima etapa es la salida
endmodule

```

Para WIDTH=8, esto genera 3 etapas ($\log_2 8 = 3$):

- Etapa 0: si $i_shift[0]=1$, shift 1 posición
- Etapa 1: si $i_shift[1]=1$, shift 2 posiciones
- Etapa 2: si $i_shift[2]=1$, shift 4 posiciones

Combinándolas se puede hacer cualquier shift de 0 a 7 posiciones en solo 3 niveles de muxes.



3.4 Ejemplo: Cadena de registros (shift register)

Otro uso clásico de generate: crear una cadena de flip-flops para delay/pipeline:

```

module delay_chain #(
    parameter int WIDTH = 8,
    parameter int STAGES = 4
) (
    input  logic      i_clk,
    input  logic      i_rst,
    input  logic [WIDTH-1:0] i_data,
    output logic [WIDTH-1:0] o_data
);
    logic [WIDTH-1:0] pipe [0:STAGES-1]; // array de registros

    // Primera etapa: captura la entrada
    always_ff @(posedge i_clk)
        if (i_rst) pipe[0] <= '0;
        else       pipe[0] <= i_data;

```

```
// Etapas siguientes: cada una captura la anterior
genvar k;
generate
    for (k = 1; k < STAGES; k++) begin : gen_pipe
        always_ff @(posedge i_clk)
            if (i_rst) pipe[k] <= '0;
            else      pipe[k] <= pipe[k-1];
    end
endgenerate

assign o_data = pipe[STAGES-1]; // salida es la ultima etapa
endmodule
```

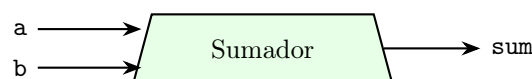
Esto genera STAGES flip-flops en serie. La señal tarda STAGES ciclos de clock en propagarse de la entrada a la salida. Se usa para pipeline, sincronización, y matching de latencia.

4 Diseño registrado: el sumador del laboratorio

4.1 ¿Por qué registrar?

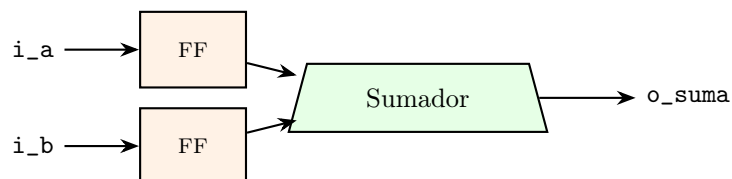
Hasta ahora nuestros sumadores eran puramente combinacionales: `assign sum = a + b`. En un chip real, las entradas y salidas se registran (pasan por flip-flops). ¿Por qué?

Sin registros (combinacional puro)



El timing depende de lo que haya
ANTES y DESPUÉS del sumador

Con registros de entrada



El timing del sumador es independiente
de lo que haya antes (los FFs lo aíslan)

Los flip-flops “cortan” el timing path. El sintetizador solo tiene que preocuparse por el delay del sumador entre los FFs de entrada y el port de salida, no por lo que haya antes.

4.2 El sumador registrado

Este es el diseño que usamos para el barrido de síntesis:

```
module sumador #(parameter int WIDTH = 8) (
    input  logic          i_clk,
    input  logic          i_rst,
    input  logic          i_enable,
    input  logic [WIDTH-1:0] i_a,
    input  logic [WIDTH-1:0] i_b,
    output logic [WIDTH:0]  o_suma // WIDTH+1 bits para el carry
);
```

```

);
  logic [WIDTH-1:0] r_a, r_b;           // registros de entrada

  always_ff @(posedge i_clk) begin
    if (i_rst) begin
      r_a <= '0;
      r_b <= '0;
    end else if (i_enable) begin
      r_a <= i_a;
      r_b <= i_b;
    end
  end

  assign o_suma = r_a + r_b;           // suma combinacional
endmodule

```

El timing path crítico es $r_a_reg / r_b_reg \rightarrow \text{sumador} \rightarrow o_suma$. Es un path **reg2out** (registro a output port).

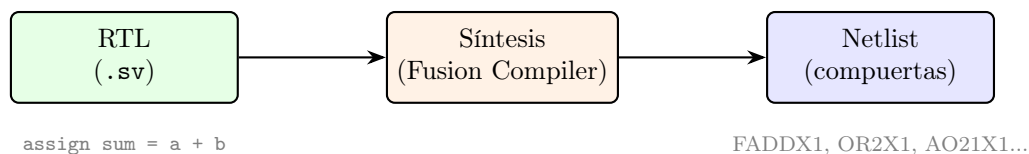
Nota sobre el ancho de la salida

La salida tiene $WIDTH+1$ bits. Si sumás dos números de 8 bits (máximo $255 + 255 = 510$), necesitás 9 bits para representar el resultado sin overflow. En general: $\text{bits}(a + b) = \max(\text{bits}(a), \text{bits}(b)) + 1$.

5 Primer contacto con síntesis

5.1 ¿Qué es síntesis?

Síntesis transforma el RTL (SystemVerilog) en un **netlist de celdas estándar**: compuertas, flip-flops, sumadores, muxes de una librería tecnológica específica.



El sintetizador decide **qué celdas usar** para implementar tu diseño cumpliendo las constraints de timing que le diste en el SDC.

5.2 SDC: las constraints

El archivo SDC le dice al sintetizador qué timing tiene que cumplir:

```

set PERIOD 1.0                               # 1ns = 1 GHz
set CONSTRAINT [expr {$PERIOD * 0.10}]       # 10% del periodo

create_clock -period $PERIOD -name clk [get_ports i_clk]
set_input_delay -clock clk $CONSTRAINT \
  [remove_from_collection [all_inputs] [get_ports i_clk]]
set_output_delay -clock clk $CONSTRAINT [all_outputs]

```

5.3 Input Delay y Output Delay

Tu diseño no vive aislado — está conectado a otros chips o bloques. Los comandos `set_input_delay` y `set_output_delay` modelan el mundo exterior.

5.3.1 Input Delay

`set_input_delay` le dice al sintetizador: “los datos que entran a mi diseño llegan X nanosegundos **después** del flanco del clock”.

¿Por qué? Porque antes de tu diseño hay otro chip (o bloque) que tiene su propio flip-flop, su lógica combinacional, y cables de PCB. Todo eso tarda un tiempo. Los datos no llegan en el instante exacto del clock — llegan un poco después.

Si el input delay es 0.1ns con un período de 1.0ns, tu diseño tiene $1.0 - 0.1 = 0.9$ ns para procesar los datos. El input delay le “quita” tiempo a tu lógica interna.

Más input delay = menos tiempo disponible para tu diseño = más difícil cerrar timing.

5.3.2 Output Delay

`set_output_delay` le dice al sintetizador: “el chip que recibe mi salida necesita que los datos estén listos X nanosegundos **antes** del próximo flanco del clock”.

¿Por qué? Porque después de tu diseño hay cables, lógica combinacional, y un flip-flop en el chip receptor que tiene su propio T_{setup} . Necesita que el dato llegue con anticipación.

Si el output delay es 0.1ns con un período de 1.0ns, tu diseño tiene $1.0 - 0.1 = 0.9$ ns para producir la salida. El output delay también le quita tiempo.

Más output delay = menos tiempo para generar la salida = más difícil cerrar timing.

5.3.3 En el SDC

```
set PERIOD 1.0
set CONSTRAINT [expr {$PERIOD * 0.10}] # 10% del periodo = 0.1ns

# Input delay: 0.1ns despues del clock llegan los datos
set_input_delay -clock clk $CONSTRAINT \
    [remove_from_collection [all_inputs] [get_ports i_clk]]

# Output delay: 0.1ns antes del proximo clock deben estar los datos
set_output_delay -clock clk $CONSTRAINT [all_outputs]
```

Con 10% de constraint en ambos lados, tu diseño tiene el 80% del período para su lógica interna (10% se lo come el input delay, 10% el output delay).

6 Resumen

Concepto	Lo esencial
Module	La unidad de diseño. Tiene ports (entrada/salida) y lógica interna.
Parámetros	Hacen el module reutilizable. Se definen con <code>parameter</code> y se configuran al instanciar con <code>#(.PARAM(valor))</code> .
Instanciación	Usar un module dentro de otro: <code>modulo inst (.port(signal))</code> . Cada instancia es hardware independiente.
Jerarquía	Modules dentro de modules. Permite construir diseños complejos a partir de bloques simples.
Generate for	Crea múltiples instancias con un loop. Se resuelve en compilación, no en runtime. Necesita <code>genvar</code> y label.
Registros de entrada	Los flip-flops “cortan” timing paths. Aíslan bloques entre sí.
Síntesis	Transforma RTL en compuertas. El sintetizador elige la implementación según las constraints de timing.